Rapport de stage

Rendre visible l'invisible des communications dans un système multi-agents

Anne REGENNASS

Année 2025–2026

Stage de deuxième année réalisé dans le laboratoire du CRAN

Maître de stage : Pascale MARANGÉ

Encadrant universitaire : Zahra RONDEAU

Déclaration sur l'honneur de non-plagiat

Je soussigné(e),

Nom, prénom : Regennass, Anne

Élève-ingénieur(e) régulièrement inscrit(e) en 2e année à TELECOM Nancy

Numéro de carte de l'étudiant(e) : 32417956

Année universitaire : 2025–2026

Auteur(e) du document, mémoire, rapport ou code informatique intitulé :

Rendre visible l'invisible des communications dans un système multi-agents

Par la présente, je déclare m'être informé(e) sur les différentes formes de plagiat existantes et sur les techniques et normes de citation et référence.

Je déclare en outre que le travail rendu est un travail original, issu de ma réflexion personnelle, et qu'il a été rédigé entièrement par mes soins. J'affirme n'avoir ni contrefait, ni falsifié, ni copié tout ou partie de l'œuvre d'autrui, en particulier texte ou code informatique, dans le but de me l'accaparer.

Je certifie donc que toutes formulations, idées, recherches, raisonnements, analyses, programmes, schémas ou autres créations, figurant dans le document et empruntés à un tiers, sont clairement signalés comme tels, selon les usages en vigueur.

Je suis conscient(e) que le fait de ne pas citer une source ou de ne pas la citer clairement et complètement est constitutif de plagiat, que le plagiat est considéré comme une faute grave au sein de l'Université, et qu'en cas de manquement aux règles en la matière, j'encourrais des poursuites non seulement devant la commission de discipline de l'établissement, mais également devant les tribunaux de la République Française.

Fait à Goncelin, le 1er septembre 2026



Signature :

Rapport de stage

Rendre visible l'invisible des communications dans un système multi-agents

Anne REGENNASS

Année 2025–2026

Stage de deuxième année réalisé dans le laboratoire du CRAN

Anne REGENNASS
34 route de Pelane
38570, Goncelin
+33(0)6 95 48 45 11
anne.regennass@gmail.com

TELECOM Nancy
193 avenue Paul Muller,
CS 90172, VILLERS-LÈS-NANCY
+33 (0)3 83 68 26 00
contact@telecomnancy.eu

CRAN
745, Rue du Jardin Botanique
54600, Villers-lès-Nancy
+33(0)3 72 74 00 00

Maître de stage : Pascale MARANGÉ

Encadrant universitaire : Zahra RONDEAU

Remerciements

Je tiens à remercier Pascale Marangé pour m'avoir proposé ce stage et pour m'avoir encadrée pendant tout son déroulement.

Je remercie également Pascale Marangé et Rémi Pannequin pour leurs retours, leurs conseils, leur accompagnement et leur bienveillance tout au long de ce stage.

À l'ensemble de l'équipe de l'AIP Lorrain pour leur accueil chaleureux tout au long de ces deux mois de stage.

À Jennifer et Jean-Luc pour leur intérêt et conseils.

À Zahra Rondeau pour son suivi tout au long de ce stage.

Et enfin à Raphaël Estivals pour avoir effectué ce stage avec moi, pour son écoute et son entraide.

Table des matières

Remerciements **iv**

Table des matières **v**

1 Introduction **1**

2 Présentation du Laboratoire d'Accueil **3**

2.1 Le CRAN : Centre de Recherche en Automatique de Nancy 3

2.2 L'Équipe MPSI : Modélisation, Pilotage et Sûreté des Systèmes Industriels 3

3 Problématique et Objectif du Stage **5**

3.1 Problématique 5

3.2 Objectif du Stage 5

3.3 Connaissances et Travail Préliminaire 5

4 Travail Réalisé **7**

4.1 Reprise du travail réalisé précédemment 7

4.2 La mise en place du passthrough 7

4.3 Gestion Temporelle des Messages Agents 8

4.4 Le Positionnement dans l'Espace 9

4.4.1 Une première possibilité : La Reconnaissance d'Environnement 10

4.4.2 La solution implémentée : Un positionnement manuel 10

4.4.3 Contrer la dérive spatiale 11

4.4.4 Extension possible 11

4.5 Rendre le casque indépendant de l'ordinateur 11

4.5.1 Parenthèse sur le Cast 12

4.6 Menu 12

4.6.1 Interactivité 12

4.6.2 Panneau de Paramètres 13

4.6.3 Panneau de Légende 14
4.6.4 Affichage des messages de Logs 16
4.7 Extensions Possibles 17

5 Conclusion **18**
5.1 Pistes d'améliorations 18

Bibliographie **19**

Table des figures **20**

Résumé **21**

Abstract **22**

Signatures **23**

1 Introduction

Depuis maintenant quelques années, le monde de l'industrie se heurte à de nouvelles problématiques, en particulier liées au contexte écologique actuel. En effet, le réchauffement climatique et l'épuisement des ressources prennent de plus en plus d'importance et les industriels se doivent de les intégrer au sein de leurs chaînes de production. Ces préoccupations grandissantes s'accompagnent d'un développement de nouveaux outils liés aux nouvelles technologies. Ils ont pour vocation d'augmenter les vitesses de production et les rendements, mais aussi et surtout, d'intégrer de nouveaux processus au sein des chaînes de production.

Le projet de la plateforme ProGreSS 4.0 s'inscrit dans cette démarche de renouveau de l'industrie, en mettant au centre de ces préoccupations le concept de régénération. La régénération regroupe l'ensemble des processus ayant pour objectif de donner une seconde vie à un produit ou à ses composants et ainsi présenter des performances équivalentes à celles du produit d'origine compte tenu de l'usage prévu. [1] En effet, ce projet a pour objectif de mettre en place une chaîne de production pour revaloriser des produits. Ces produits sont les composantes d'un moteur simple, composé d'un rotor et d'un stator, puis inséré dans un carter. Un carter est une enveloppe rigide et étanche qui protège des pièces mécaniques en mouvement. [6] Cette chaîne a donc pour objectif de démonter des produits reçus pour récupérer leurs composants, les analyser pour déterminer leur état, puis d'utiliser ensuite ces pièces pour recréer un nouveau produit en fonction d'un besoin client.

Cette plateforme est composée d'un Système Flexible de Production (SFP), composé de six postes, chacun ayant un rôle différent dans notre chaîne de production. À cela s'ajoute un Magasin Auto (MagAuto) dont le rôle est de stocker les pièces et produits, ainsi que trois robots mobiles dont le rôle est de faire le lien entre les différents postes du SFP et le MagAuto. Tous ces éléments sont régis par un système multi-agents. Un système multi-agents est un système composé d'agents, soit des logiciels qui fonctionnent de façon autonome, actifs dans un certain environnement et qui interagissent entre eux selon certaines règles. [7]

La Programmation Orientée Agent (ou POA) est ainsi utilisée, selon la norme FIPA et avec Jade, une version adaptée du langage Java. Ce système multi-agents fonctionne donc et évolue du fait des échanges qui ont lieu entre les différents agents. Ces échanges peuvent prendre la forme de messages, de requêtes, de questions, de simples données ou autres, et représentent le cœur du déroulement des différentes actions des agents présents. Cependant, là se présente le principal désavantage d'un tel système : il devient vite opaque du fait de la multiplication de ces communications.

Cela peut représenter un simple ralentissement de la compréhension pour un ingénieur initié, mais cela peut rapidement devenir un réel obstacle à la compréhension pour un œil extérieur. Cela devient alors une réelle préoccupation, en particulier dans ce contexte de développement et de changements dans l'industrie, accompagné d'une volonté de mettre ces changements à la disposition des

étudiants. En effet, un des objectifs de cette plateforme est de donner la possibilité à des étudiants de l'Université de Lorraine de découvrir, comprendre et se sensibiliser à ces nouveaux moyens de production, ancrés dans ce contexte environnemental.

Ici se pose alors la principale problématique de notre stage, à Raphaël ESTIVALS et moi-même, qui est de tenter de rendre le fonctionnement de ce système multi-agents plus intelligible pour un œil extérieur, en affichant de manière instinctive les communications entre les différents agents, mais aussi de faciliter/de fluidifier son observation pour un œil interne, en soulignant les différents événements au sein de chaque agent.

C'est donc au sein de ce sous-projet de "Rendre visible l'invisible" que nous sommes intervenus pour notre stage. Là où Raphaël ESTIVALS avait pour mission de rendre compréhensibles les comportements internes des agents, notamment avec une mise en place de logs dans l'ensemble du code source, mon rôle s'est plus tourné vers l'affichage des communications entre les différents agents du système, en utilisant la réalité augmentée.

2 Présentation du Laboratoire d'Accueil

2.1 Le CRAN : Centre de Recherche en Automatique de Nancy

Principalement basé sur le campus des Sciences à Villers-lès-Nancy, le laboratoire du CRAN, ou le Centre de Recherche en Automatique de Nancy, est une unité de recherche entre l'Université de Lorraine et le CNRS. Ses préoccupations se répartissent dans plusieurs domaines, notamment ceux du traitement du signal et des images, du contrôle et du génie informatique, mais aussi dans le domaine de la biologie et des neurosciences.

Il est organisé en trois départements :

— CID - Contrôle Identification Diagnostic

— BioSiS - Biologie, Signaux et Systèmes en Cancérologie et Neurosciences

— MPSI - Modélisation, Pilotage et Sûreté des Systèmes Industriels

Son but est notamment d'accompagner les évolutions de la société et de dépasser les problématiques industrielles classiques en prenant en compte les enjeux actuels et leur évolution, notamment dans les questions autour du développement durable. [3]

2.2 L'Équipe MPSI : Modélisation, Pilotage et Sûreté des Systèmes Industriels

L'équipe que j'ai pu intégrer au cours de ces deux mois appartient au département MPSI du laboratoire, ou Modélisation, Pilotage et Sûreté des Systèmes Industriels. Ce département a pour principale préoccupation la place des dimensions humaine, technique et environnementale dans les nouveaux systèmes de production. Ils travaillent sur des problématiques liées à l'adaptabilité, la durabilité, le recyclage ou encore la résilience dans le cadre de l'industrie du futur. [4]

J'ai été accueillie au sein de l'AIP Lorrain, lieu de développement de la plateforme ProGreSS 4.0, et encadrée par Pascale MARANGÉ, enseignante-chercheure au sein de cette équipe. Elle supervise le projet "Rendre visible l'invisible", sous-projet de ProGreSS 4.0 qui est le cadre de notre stage. Ce sous-projet a pour objectif de rendre accessible le fonctionnement de la plateforme au plus grand nombre, en particulier dans un cadre pédagogique. J'ai également été encadrée par Rémi PANNEQUIN et Jean-Luc METZGER, ingénieurs de recherche, chargés de la mise en place et de la maintenance du système multi-agents, et par Jennifer MAYO SOLIS, administratrice systèmes et

réseaux, automatisation et responsable du matériel de réalité augmentée.

3 Problématique et Objectif du Stage

3.1 Problématique

Comme vu précédemment dans l'introduction 1, le projet ProGreSS 4.0 se positionne dans notre contexte actuel en le mettant au cœur de ses objectifs et préoccupations. Il représente une porte d'entrée à toutes les considérations écologiques et enjeux dans nos moyens de production. Il est alors essentiel de permettre une plus grande compréhension de son fonctionnement afin de le rendre accessible au plus grand nombre.

Cependant, étant un système multi-agents, son fonctionnement est souvent dense, surchargé en informations. Il est alors difficile pour quelqu'un d'extérieur ou de non initié de comprendre l'ensemble des actions qui se produisent simultanément. Pour rendre cette plateforme à la portée de tous, en passant des étudiants aux ingénieurs-chercheurs, une "vulgarisation" des échanges d'informations est ainsi nécessaire. De plus, la densité de ces systèmes peut également rendre leur maintenance et évolution difficile pour les ingénieurs en charge, en particulier avec une très grande quantité de messages de log (messages indiquant des erreurs, des warnings, ou simplement des points d'attention signalés au développeur). Trier ces informations, savoir de quel agent provient le message et son importance sont aussi des informations décisives et il est nécessaire de les afficher de manière efficace et instinctive.

3.2 Objectif du Stage

Ce stage, réalisé avec Raphaël ESTIVALS, également étudiant de seconde année à TELECOM Nancy, a donc regroupé plusieurs objectifs. Là où Raphaël ESTIVALS a été chargé de permettre une meilleure compréhension des comportements des agents en développant un système de logs dans l'intégralité du programme déjà présent, le mien a été d'afficher, grâce à la réalité augmentée, les échanges d'informations entre les différents agents de la plateforme. Cet affichage doit être pédagogique, facilement compréhensible, tout en regroupant l'intégralité des informations importantes.

3.3 Connaissances et Travail Préliminaire

Dans le cadre de mon cursus à TELECOM Nancy, j'ai eu l'occasion, au cours du second semestre de seconde année, de réaliser un projet en laboratoire dans le cadre du PIDR (Projet Interdisciplinaire et/ou de Découverte de la Recherche). En duo avec Bleunvenn AUDREN, nous avons eu le plaisir

de travailler sur le sujet de “Rendre visible l’invisible des communications entre agents grâce à la réalité augmentée”. Ce sujet a pu donner suite à ce stage, qui s’inscrit alors dans la continuité du travail déjà effectué.

Au cours du PIDR, nous avons eu l’occasion de découvrir le logiciel Unity, ainsi que le fonctionnement de la Plateforme ProGreSS 4.0 et de son environnement. Il a été mis en place un programme permettant de lire des fichiers de traces de messages, et de les afficher de la manière suivante : les agents sont représentés par des cubes et les messages par des sphères qui circulent ainsi d’un agent à un autre, entre la source et la destination du message agent. De plus, les agents robots détiennent une carte commune contenant tous les points clés de l’environnement dans lequel ils évoluent. Un programme a été réalisé permettant de récupérer ces données et de les afficher à l’échelle dans Unity en utilisant les coordonnées récoltées dans ce fichier de carte.

À la fin de ce projet, plusieurs éléments clés ont été définis, notamment la spatialité des échanges et leur temporalité, notions qui ont été abordées au début de ce stage.

4 Travail Réalisé

4.1 Reprise du travail réalisé précédemment

Le travail réalisé lors du PIDR a servi de base de départ à ce stage. Une première partie a donc été de reprendre le code réalisé pour en comprendre son intégralité, déterminer ses failles et points d'améliorations. Trois points principaux en sont ressortis :

- L'absence de passthrough. Le passthrough est la capacité, pour un casque de réalité augmentée, de capturer des images réelles et de les retransmettre ensuite sur ses écrans. C'est cette capacité qui permet, entre autres, la réalité mixte, c'est-à-dire l'affichage d'éléments 3D virtuels dans un environnement réel. [2]
- L'absence de gestion temporelle des messages agents affichés et leurs changements d'états
- L'absence de synchronisation spatiale entre la carte 3D réalisée et la réalité

Ils représentent donc les principaux points de départ de travail de ce stage. À ceux-ci se rajoutent d'autres points établis au fur et à mesure de l'avancement du stage et des tâches réalisées.

4.2 La mise en place du passthrough

Le passthrough est la capacité pour un casque de réalité augmentée d'afficher à la fois une retranscription de la réalité mais également des éléments graphiques 3D. C'est une fonctionnalité phare du projet qui n'a pas pu être mise en place au cours du PIDR du fait du manque de connaissances du logiciel Unity. Pour permettre cet affichage mixte, nous devons utiliser un objet dans Unity qui va permettre la gestion de la caméra du casque. Le choix de la bibliothèque à utiliser est alors déterminant puisque de celui-ci va dépendre la mise en place d'autres fonctionnalités que l'on pourrait choisir d'implémenter plus tard. Deux choix s'offrent à nous :

- Meta XR SDK : c'est la bibliothèque développée par Meta pour ses casques de réalité virtuelle et augmentée. De ce fait, toutes les fonctionnalités des casques Meta sont immédiatement exploitables, et Meta propose un support régulier. Cependant, cela limite voire supprime la portabilité du projet sur d'autres matériels hors écosystème Meta.
- Open XR : il s'agit de la bibliothèque standard de Unity, adaptée à tous les supports. Cela rend le portage vers d'autres plateformes ou matériel très facile. Cependant, cela peut ajouter quelques difficultés pour implémenter ou utiliser des fonctionnalités propres à notre casque.

Dans notre cas, nous disposons d'un casque Oculus Meta Quest 3. Pour faciliter l'utilisation de certaines fonctionnalités du casque, comme l'ancrage spatial, et car l'utilisation d'autres supports

n'est pas envisagée au sein de l'AIP, il a été choisi d'utiliser la librairie Meta XR SDK pour la suite de ce projet.

Une fois la librairie choisie, il suffit d'ajouter dans notre projet un objet précis prenant en charge la gestion du casque et de sa caméra : l'OVRCameraRig. Après paramétrage de l'objet, le pass-through est fonctionnel et nous pouvons nous concentrer sur la gestion des éléments à afficher.

4.3 Gestion Temporelle des Messages Agents

Au sein du système, il existe un agent particulier nommé le Relation Sniffer dont l'objectif est d'observer et récolter, de "sniffer", tous les échanges d'informations ayant lieu entre les différents agents. Lors d'une activité sur la plateforme, il va donc récolter les différentes informations qui circulent et les enregistre dans un fichier YAML que l'on appelle une trace. Cette trace garde la nature de chaque message et le temps qui s'est écoulé entre le lancement du système et l'envoi de celui-ci.

Le programme mis en place au cours du PIDR utilisait un fichier trace donné et jouait les interactions sans prendre en compte le temps : les actions étaient jouées à intervalle de temps régulier. Un premier objectif était donc de respecter la temporalité des événements.

Or, nous disposons d'un outil pour ajouter cette dimension temporelle. En effet, il existe un programme créant un serveur REST qui, pour une trace donnée, va comme "rejouer" les événements en respectant ses intervalles de temps. Il nous suffit donc de lancer puis d'interroger ce serveur pour récupérer les événements au fur et à mesure qu'ils sont joués. Ce serveur renvoie plusieurs données sous forme de quatre listes :

- Les agents présents
- Les relations actives entre eux
- Les messages Agents échangés
- Les messages de logs, que l'on détaillera plus tard dans ce rapport

À ces quatre listes s'ajoute un entier qui représente le numéro de l'événement qui est envoyé. Ce numéro est essentiel puisqu'il permet au programme de ne pas afficher plusieurs fois les mêmes messages agents. En effet, un appel est fait au serveur toutes les 0,5 seconde ; il est possible d'appeler le serveur sans qu'aucun nouveau message n'ait été envoyé entre-temps. Cependant, le serveur va quand même renvoyer les messages agents, même si ceux-ci l'ont déjà été. Ce numéro d'événement permet donc d'identifier s'il s'agit du même message agent, auquel cas on ne l'affiche pas, ou d'un nouveau et de l'afficher.

Une fois le problème de duplication des événements réglé avec la mise en place du numéro d'événement, il ne reste plus qu'à récupérer les données envoyées avec une structure adaptée et à les afficher en adaptant le programme mis en place lors du PIDR. Des modifications esthétiques ont été effectuées au fur et à mesure du stage pour rendre l'affichage plus instinctif à la compréhension, comme par exemple des couleurs différentes en fonction du type de message agent à afficher.

La liste des messages envoyés par le serveur ne comporte pas uniquement des messages agents envoyés d'un agent à un autre, mais également des messages de télémétrie. Ces messages peuvent être multiples : ils peuvent signaler un changement d'état d'un agent, comme par exemple un changement de position d'un robot mobile, ou signaler un état notable d'un agent. Ces messages sont également traités par le programme, ce qui permet alors de modifier la position des agents mobiles dans notre scène et d'afficher des bulles de texte au-dessus de l'agent concerné pour signaler son état.

4.4 Le Positionnement dans l'Espace

Les robots mobiles présents sur la plateforme disposent d'une carte qui répertorie tous les points clés de l'espace :

- la délimitation de leur zone d'action, avec des lignes ou des zones interdites, qu'ils ne peuvent traverser
- la position des docks de charge de tous les robots mobiles
- la position de certains points objectifs, comme celles des différents postes du SFP et du Magasin Auto, avec lesquels ils peuvent interagir, ou comme l'entrée de certaines salles de l'AIP.

Cette carte est commune à tous les robots mobiles présents sur la plateforme.

Au cours du PIDR, un traitement de cette carte a été effectué. En effet, un programme a été mis en place afin de récupérer les données présentes dans ce fichier pour en afficher les éléments dans Unity. On se retrouve donc avec une reproduction de la carte sur notre machine qui fonctionne avec son propre repère local, que nous appellerons repère virtuel dans la suite de ce rapport. Les distances et proportions du repère virtuel sont respectées à un facteur près puisque le fichier de carte comporte les coordonnées de tous les éléments qui y sont répertoriés.

Lorsque l'on souhaite afficher cette scène virtuelle dans le casque, on se heurte à un nouveau problème : à chaque fois que l'on lance le programme d'affichage, le repère virtuel, l'affichage de nos éléments 3D, ne s'affiche pas au même endroit. En effet, la position dans notre réalité de l'origine du repère de la scène virtuelle est calculée par rapport aux coordonnées du casque de réalité augmentée. À chaque fois que le programme est lancé, la position courante du casque est utilisée comme point d'origine. Donc il suffit de se positionner différemment au démarrage pour que les objets 3D apparaissent à des endroits différents. Cependant cela représente un réel problème pour notre affichage puisque si les éléments 3D ne correspondent pas à notre réalité, les échanges d'informations entre les agents deviennent incompréhensibles, puisque mal positionnés. Une nécessité de positionner ce repère apparaît donc.

4.4.1 Une première possibilité : La Reconnaissance d'Environnement

L'une des fonctionnalités dont dispose le casque Oculus Meta Quest 3 est la reconnaissance de l'environnement. En effet, théoriquement, il est possible d'utiliser le casque pour reconnaître des éléments distinctifs d'un environnement et de pouvoir alors utiliser cet élément comme repère dans l'espace. En effet, si on connaît la position d'un élément caractéristique dans notre repère virtuel, et en trouvant cet élément dans la pièce réelle, on peut alors calculer une origine pour le repère virtuel et bien le synchroniser sur le réel.

Cette piste a donc été explorée avec une peinture sur un mur de l'AIP, puis avec un panneau avec des pictogrammes. Ce changement de point de repère a été effectué pour utiliser un repère avec des couleurs plus vives et des changements de couleurs plus francs. Cependant, la détection d'un repère via le casque a échoué : il est possible que cela soit dû à une lumière irrégulière ou insuffisante dans le lieu, à des reflets sur le repère le rendant indétectable, ou à un élément de repère pas assez distinctif. Après plusieurs tentatives, cette possibilité a donc été écartée.

4.4.2 La solution implémentée : Un positionnement manuel

Puisque nous disposons des coordonnées virtuelles des éléments 3D grâce à la carte des robots, nous pouvons utiliser ces données pour positionner notre repère. En effet, si on connaît les coordonnées réelles (les coordonnées via le repère du casque) d'au moins deux objets, on peut alors calculer la position de l'origine virtuelle (celle de la carte) dans le repère du casque. Prendre deux points permet également de déterminer précisément l'orientation de ce repère. Une phase de synchronisation a donc été ajoutée lors du lancement du programme : un message d'instruction apparaît au démarrage indiquant à l'utilisateur de se positionner au niveau du dock de charge d'un des robots et de cliquer sur un bouton de sa manette. Lorsque le bouton est actionné, sa position est récupérée et enregistrée. On réitère l'opération au niveau d'un autre dock de charge et on obtient nos points de coordonnées dans le réel. On peut alors les utiliser et calculer notre origine.

Maintenant que notre nouvelle origine est calculée, il faut ensuite appliquer cette origine à tous nos éléments 3D que l'on veut afficher. C'est à ce moment que l'organisation de notre hiérarchie Unity entre en scène. En effet, dans Unity, la hiérarchie correspond à l'organisation de nos objets dans la scène. Elle regroupe tous nos éléments 3D, mais aussi la caméra du casque de réalité augmentée. On va alors créer un objet qui va servir de parent à tous les objets du repère 3D. Il est cependant essentiel de ne pas inclure la caméra dans cet objet parent. En effet, positionner la caméra sous ce parent changerait également l'origine du casque et la synchronisation ne serait alors pas correcte.

Lorsque l'on crée ensuite des objets 3D, il suffit d'indiquer que leur point de repère est cet objet parent, ce qui permet d'automatiquement les positionner en fonction de celui-ci.

4.4.3 Contrer la dérive spatiale

Maintenant que la scène est synchronisée avec la réalité, un autre problème peut apparaître : celui de la dérive spatiale. En effet, le casque recalcule en permanence les mouvements de l'utilisateur, comme ses déplacements, sa vitesse, sa rotation, pour savoir en permanence la position du casque par rapport à sa position initiale. Cependant, ces calculs permanents peuvent être à l'origine d'erreurs, et bien qu'elles soient minimes, elles peuvent s'accumuler, et devenir suffisamment grandes pour fausser la position du casque et décaler tout l'affichage 3D. Pour contrer cette dérive spatiale, une solution simple est de recalculer notre origine. Ainsi, en réactivant la phase de synchronisation détaillée ci-dessus, on recalcule une nouvelle origine qui prend en compte les changements nés de la dérive spatiale, et le problème est évité. De plus, cela permet à l'utilisateur de changer la position du repère si une erreur ou fausse manipulation s'est produite au moment de la phase de synchronisation.

Il faut donc mettre en place une interface permettant de déclencher cette phase de resynchronisation. Nous détaillerons son implémentation ci-après, en section 4.6.2.

4.4.4 Extension possible

Une autre piste a été étudiée concernant la dérive spatiale mais non mise en place par manque de temps. Le casque utilisé permet la mise en place d'ancres spatiales. Ces objets permettent de mettre en place un point de repère, une "ancre", dans un environnement que le casque va enregistrer. Puisque sa position est connue par le casque, lorsqu'il va reconnaître l'environnement en question, il va automatiquement se recalibrer en fonction de son ancre.

Ces ancres peuvent également être enregistrées et donc conservées d'une utilisation à une autre du casque, ce qui permettrait de fluidifier l'utilisation du casque en ne réalisant la phase de synchronisation qu'à la demande de l'utilisateur et non plus systématiquement. Cependant, les tests de reconnaissances de l'environnement présentés plus tôt n'ayant pas été fructueux, la mise en place d'ancres spatiales n'a pas pu être effectuée.

4.5 Rendre le casque indépendant de l'ordinateur

Au début du projet, le casque dépendait beaucoup de la machine sur laquelle était créé le programme. En effet, avec Unity, il est possible de compiler le projet et son code sur l'ordinateur et d'observer le résultat dans le casque grâce à un lien "Link", filaire, ou "Air Link", par connexion WiFi. L'avantage de cette méthode est que le programme tourne sur l'ordinateur et cela réduit donc le temps de compilation et donc d'attente pour effectuer les tests. Cependant, à terme, le souhait est que le casque fonctionne de lui-même, c'est-à-dire qu'il puisse fonctionner indépendamment de la machine qu'on utilise. Cela permet d'obtenir des tests plus fiables, puisqu'ils dépendent des capacités du casque et non pas de la machine, en particulier dans notre cas où l'affichage est crucial

puisqu'on utilise la puce du casque et non plus le processeur et la carte graphique de la machine.

Changer notre façon de compiler le programme et de tester le projet de manière différente a permis de faire naître de nouveaux besoins, notamment celui de permettre à l'utilisateur de pouvoir changer dans le casque certains paramètres de nos fonctions, et en particulier l'adresse du serveur que l'on veut interroger. Ces nouveaux besoins ont mené à la conception d'un panneau de paramètres que l'on détaillera dans la section 4.6.2.

4.5.1 Parenthèse sur le Cast

L'un des avantages de cette ancienne connectivité Link entre le casque et l'ordinateur était de permettre d'avoir un retour des images présentes dans le casque sur le moniteur de notre machine. Cela présentait un avantage considérable puisque cela permettait à d'autres personnes, autres que l'utilisateur du casque, d'observer l'affichage 3D. Cependant, cette fonctionnalité peut être remplacée par un cast du casque lors de son utilisation. Un cast, screencast, ou mise en miroir, est l'enregistrement vidéo ou la retransmission d'un écran sur un autre. [5] Dans notre cas, de l'écran du casque sur un ordinateur.

En effet, de nombreux outils sont mis à disposition par Meta en ligne pour permettre le cast du casque lors de son utilisation. Cette problématique est donc évitée.

4.6 Menu

Au fur et à mesure du travail réalisé, la nécessité de mettre en place un menu s'est de plus en plus imposée. En effet, pour rendre le casque réellement indépendant de l'ordinateur, pour permettre la resynchronisation du casque, ou pour permettre une meilleure compréhension des affichages 3D, une interface interactive regroupant tous ces éléments devenait nécessaire.

4.6.1 Interactivité

Le premier problème rencontré pour la mise en place de ce menu a été d'intégrer de l'interactivité. En effet, pour rendre le menu ergonomique pour l'utilisateur, il est essentiel de lui permettre d'interagir avec les affichages qui lui sont proposés. Car si faire apparaître une fenêtre à la suite d'un clic sur un bouton de la part de l'utilisateur se met en place rapidement, ajouter une interactivité au niveau du panneau n'est pas chose aisée. Pour ajouter cette fonctionnalité, il a été choisi d'utiliser un préfabriqué Unity appelé FlatUnityCanvas. En effet, ce préfabriqué contient une fenêtre, associée à une surface, et c'est cette dernière qui va permettre de rendre la fenêtre "palpable". La surface est la partie solide de notre objet. Ce préfabriqué contient également des boutons avec lesquels nous pouvons interagir. Cet objet va donc représenter notre barre de navigation entre nos différents panneaux, visible en figure 4.1. Il est ainsi possible de naviguer dans cette barre en faisant

défiler les boutons de gauche à droite.

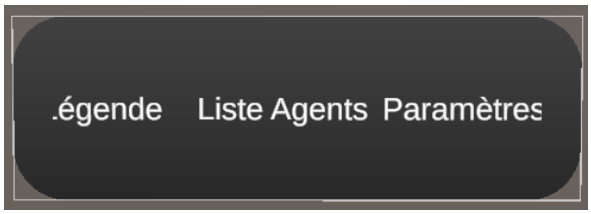


FIGURE 4.1 – Barre de sélection des panneaux

Des préfabriqués similaires vont ensuite être utilisés pour créer les panneaux de notre menu. La barre de boutons va donc permettre de naviguer entre ces différents panneaux que nous allons détailler ci-après.

Parenthèse sur les Building Blocks

Une autre possibilité explorée a été l'utilisation des Building Blocks. Ce sont des ensembles d'objets déjà construits et paramétrés permettant de mettre en place des fonctionnalités précises, comme la gestion des mains et des manettes, la mise en place de surfaces palpables, etc. Cependant, ces Building Blocks fonctionnent ensemble et dépendent donc les uns des autres, et l'un des blocs essentiels à la mise en place des autres est celui de gestion de la caméra. Or, étant des objets intégrés par Unity, ces Blocks utilisent la librairie Open XR écartée précédemment. L'utilisation des Building Blocks disponibles dans Unity a donc été abandonnée du fait de cette nécessité de changement de librairie.

4.6.2 Panneau de Paramètres

Le panneau de paramètres mis en place doit regrouper plusieurs fonctionnalités clés.

Permettre le Lancement de la Resynchronisation

Comme indiqué précédemment en section 4.4.3, la dérive spatiale implique le besoin de permettre à l'utilisateur de déclencher une nouvelle phase de synchronisation de l'espace. Pour ce faire, il faut donc ajouter à notre panneau un bouton permettant de relancer cette phase, qui doit cacher les éléments 3D déjà affichés, donner les instructions à l'utilisateur et enfin recalculer la nouvelle origine du repère pour repositionner nos objets 3D. Il n'est pas nécessaire de supprimer les objets 3D déjà présents pour ensuite les recréer à une position différente. En effet, la mise en place d'un parent comme indiqué plus tôt, permet de ne modifier que celui-ci. Ses changements vont automatiquement influencer ses enfants et donc positionner nos éléments 3D correctement.

Permettre le Changement d'Adresse Réseau

Comme annoncé plus tôt, il est nécessaire de donner à l'utilisateur la possibilité de changer certains paramètres du code, et en particulier de pouvoir entrer une nouvelle adresse pour le serveur.

L'utilisateur doit donc pouvoir écrire une nouvelle URL. Nous devons donc mettre en place un clavier. Celui-ci sera lié à un objet nommé un InputField qui, en cliquant dessus, nous permet de taper du texte et de voir notre chaîne de caractères évoluer. Une fois la chaîne complète, nous pouvons utiliser cette nouvelle URL pour effectuer nos appels au serveur.

Le clavier, visible en figure 4.2, est donc composé de simples boutons qui donnent leur étiquette comme texte lorsqu'on leur appuie dessus. Il n'était pas nécessaire de reproduire un clavier standard entier puisque tous les caractères ne sont pas utilisés pour l'écriture des URL. Seules les lettres, minuscules et majuscules, les chiffres et certains caractères spéciaux sont utilisés. Afin de faciliter et d'accélérer l'utilisation du clavier, des boutons de chaînes de caractères ont également été ajoutés. Ces chaînes de caractères sont celles les plus utilisées dans notre contexte d'URL à un serveur REST (par exemple "http ://", le numéro de port utilisé, ou autre). On a donc la mise en place d'un Shift pour passer de majuscule à minuscule, et de chiffre à caractères spéciaux. Le fonctionnement est que chaque bouton a ses deux caractères prédéfinis, et qu'on passe d'un à l'autre si la touche Shift est active, comme sur un clavier d'ordinateur.



Figure 4.2 – Clavier final obtenu

Une fois ces besoins remplis, nous obtenons le panneau de paramètres en figure 4.3. Il regroupe ainsi le bouton de resynchronisation de la scène, la barre permettant de changer d'URL mais également un texte affichant l'URL courante.

4.6.3 Panneau de Légende

Comme indiqué plus tôt dans ce rapport, le centre de ce projet est de rendre l'affichage de ces échanges d'informations instinctif pour n'importe quel utilisateur. Il est donc rapidement venue l'idée d'utiliser des couleurs différentes en fonction des types de messages agents échangés ou des états des agents. Un besoin de décrire cet affichage est donc apparu, et une des solutions la mieux adaptée est de mettre en place une légende. Un nouveau panneau regroupant toutes les correspondances des affichages a donc été ajouté. Vous trouverez un aperçu de ce panneau en figure 4.4.

Tous les pictogrammes de description n'ont pas pu être ajoutés par manque de temps.



FIGURE 4.3 – Panneau de paramètres

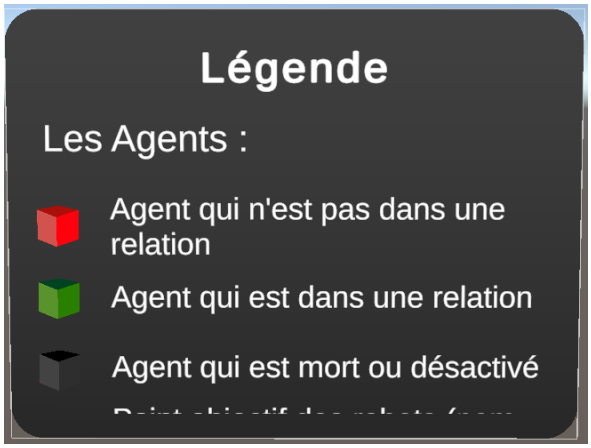


FIGURE 4.4 – Panneau de Légende

4.6.4 Affichage des messages de Logs

Grâce au travail effectué par Raphaël ESTIVALS, nous disposons de messages de *logs* envoyés également par le serveur. Ces *logs* sont structurés d'une manière définie ensemble pour répondre aux besoins suivants :

- toutes les données des *logs* doivent être conservées (l'agent dont il provient, son comportement d'origine, etc.)
- on doit pouvoir différencier chaque partie des *logs* pour les afficher facilement
- on doit savoir de quel niveau de *log* il s'agit

En effet, un nouveau critère est de définir un niveau de l'utilisateur, pour que l'on puisse afficher plus ou moins de *logs*, en fonction de si l'utilisateur en a besoin ou non. Plusieurs profils utilisateurs ont ainsi été définis :

- Profil étudiant, ou personnel enseignant externe à la plateforme : les *logs* ne sont pas utiles et ne doivent donc pas être affichés.
- Profil ingénieur hors projet : les principaux messages d'erreurs ou de *warning* peuvent s'avérer intéressants, mais les détails ne sont pas nécessaires.
- Profil développeur interne au projet : tous les messages doivent être affichés.

On peut imaginer d'autres profils intermédiaires qui demanderaient de n'afficher que certains types de messages de *log*.

Un nouveau panneau a donc été mis en place, nommé Liste Agents, qui regroupe l'ensemble des *logs*. Chaque *log* est issu du fonctionnement d'un agent en particulier auquel il est attaché. On affiche ainsi dans notre panneau chaque agent ayant au moins un *log*. L'utilisateur a la possibilité de consulter ces différents *logs* et de supprimer ceux de son choix. Chaque *log* ayant un niveau d'importance, des pictogrammes visuels ont été créés pour permettre une identification rapide de ces niveaux. Vous trouverez des exemples de pictogrammes ci-après en figures 4.5 et 4.6. Ces pictogrammes sont nécessaires pour faciliter la lisibilité du panneau et permettre ainsi à l'utilisateur d'identifier rapidement les points nécessitant son attention.



FIGURE 4.5 – Pictogramme de niveau Warning



FIGURE 4.6 – Pictogramme de niveau Info

Ce panneau n'a pas pu être testé en conditions réelles par manque de temps.

4.7 Extensions Possibles

De nombreux éléments n'ont pas pu être explorés en entier, ou pas en profondeur, par manque de temps. En effet, l'un des points qui a été abordé en réunion est la manière dont on fait appel au serveur REST. En effet, actuellement, le casque interroge le serveur à intervalles de temps réguliers pour obtenir ces informations. Cependant, une piste à explorer serait de permettre, par exemple, au serveur d'envoyer de lui-même les données dès qu'un nouvel événement se produit.

Une autre piste non explorée serait de permettre à l'affichage de s'effectuer en temps réel. En effet, actuellement, le serveur rejoue une trace qui lui est donnée, mais ne fonctionne pas en temps réel avec la plateforme. Cependant, changer son fonctionnement pour permettre un traitement en temps réel nécessite un changement plus profond de son fonctionnement, et du fonctionnement d'autres agents du système, notamment du Relation Sniffer, ce qui a donc été écarté pour ce stage.

5 Conclusion

Ce stage avait pour objectif de montrer le fonctionnement de la plateforme au plus grand nombre, en adaptant les données à un affichage graphique simple et instinctif. Plusieurs objectifs ont été atteints, notamment le placement spatial des éléments 3D, la gestion temporelle de l'apparition des éléments et l'ajout d'une légende pour en faciliter la compréhension. Cependant, ce projet n'est pas encore abouti, et de nombreuses pistes restent à améliorer ou à créer, notamment l'affichage des logs, la finalisation de la légende, ou encore le modèle 3D pour l'affichage des messages dont une nouvelle version a commencé à être mise en place, mais sans le temps de l'intégrer entièrement au projet.

5.1 Pistes d'améliorations

Le manque de temps a été le plus grand frein à l'ajout ou au test de nouvelles fonctionnalités, et il est probablement dû au temps que j'ai consacré à la découverte plus en profondeur du logiciel Unity et de ses packages. Car en effet, beaucoup de pistes explorées se sont avérées infructueuses, et certaines implémentations plus longues qu'elles n'auraient pu être. Cependant, cela m'a à la fois permis de mieux comprendre l'environnement dans lequel je travaillais et d'être plus à l'aise avec le logiciel Unity et son langage, le C#, qui ont tous deux été des découvertes au cours du PIDR, puis approfondis et compris au cours de ce stage.

De plus, un grand frein a également été l'organisation du projet Unity. En effet, pour limiter la multiplication des packages et librairies sur le dépôt Git du projet, il a été choisi de travailler sur deux projets Unity en parallèle : un projet bac à sable pour expérimenter, tester des librairies et des implémentations, et un projet "final", sur le Git, où les versions testées et validées du premier projet sont intégrées. Cette structure a ses avantages mais également des inconvénients de temps, puisque le passage d'un projet à un autre a, quelques fois, été fastidieux, me forçant à réitérer certaines opérations plusieurs fois. Ainsi, une plus grande connaissance de Unity et de sa prise en charge par Git aurait été nécessaire pour fluidifier ces opérations.

Aussi, une meilleure planification des objectifs pourrait permettre une meilleure gestion du temps, ainsi qu'une meilleure estimation du temps de chaque tâche. Ces éléments seront à prendre en compte dans mes futurs projets.

Bibliographie

[1] Actu Environnement. Dictionnaire environnement - régénération.

[2] Vernon Bérubé. Qu'est-ce que le passthrough en réalité virtuelle et comment façonne-t-il l'avenir de l'XR ?, 2024.

[3] CRAN, Université de Lorraine. CRAN - présentation.

[4] CRAN, Université de Lorraine. Modélisation, pilotage et sûreté des systèmes industriels - MPSI.

[5] Gabriel Dabi-Schwebel. Screencast (capture d'écran vidéo), 2014.

[6] Wikipedia, l'encyclopédie libre. Carter (mécanique), 2026.

[7] Wikipedia, l'encyclopédie libre. Système multi-agents, 2026.

1

Table des figures

4.1 Barre de sélection des panneaux 13
4.2 Clavier final obtenu 14
4.3 Panneau de paramètres 15
4.4 Panneau de Légende 15
4.5 Pictogramme de niveau Warning 16
4.6 Pictogramme de niveau Info 16

7

Résumé

3

Dans le contexte environnemental actuel, les industries évoluent et intègrent les nouvelles technologies à leur cycle de production. Le développement de cette industrie 4.0 voit naître de nouveaux concepts, comme celui de la régénération, qui est au centre du projet ProGreSS 4.0. Celui-ci a pour but de développer une chaîne de production autour de ce concept et de la rendre accessible à tous afin de sensibiliser à ces problématiques actuelles. Le travail développé ici a pour but de rendre transparent le fonctionnement de cette plateforme. Car en effet, elle est régie par un système multi-agents complexe. La réalité augmentée est alors utilisée afin de faciliter la compréhension des échanges entre agents ainsi que leur fonctionnement avec la mise en place de messages de log. Grâce au travail réalisé sur Unity, un programme permet d'afficher intuitivement les messages agents échangés, tout en permettant à l'utilisateur d'avoir accès à la liste des logs par agent. 3

Mots-clés : Système multi-agents, Réalité augmentée, Industrie 4.0, Régénération 12

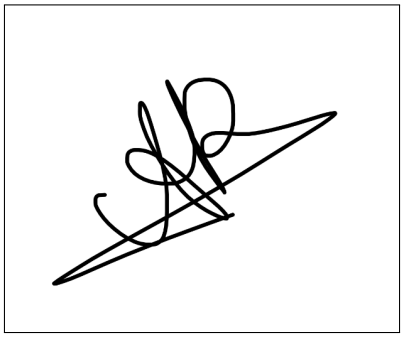
Abstract

1

In the current environmental context, industries are evolving and integrating new technologies into their production cycles. The development of Industry 4.0 has led to the emergence of new concepts, such as regeneration, which is at the heart of the ProGreSS 4.0 project. This project aims to develop a production line centered around this concept and make it accessible to everyone in order to raise awareness of these current issues. The work presented here aims to make the operation of this platform transparent, as it is governed by a complex multi-agent system. Augmented reality is used to facilitate the understanding of interactions between agents and their operation through the implementation of log messages. Thanks to the work carried out on Unity, a program makes it possible to intuitively display the messages exchanged between agents, while allowing the user to access the log history for each agent. 1

Keywords : Multi-agent system, Augmented reality, Industry 4.0, Regeneration

Signatures



Anne REGENNASS

Élève-ingénieure